



SOFTWARE PROJECT MANAGEMENT

Subject presented by:

PhD. Trần Khánh Dung

Department of Software Engineering

Email: *dungtk@huce.edu.vn*

01- 2017

Interest

- Basic software project management concepts and principles
- Process and project metrics
- Basis for effective management decision making
- Techniques used to
 - estimate cost,
 - resource requirements,
 - and establish an effective project plan
- Management activities that lead to effective risk monitoring, mitigation, and management
- Activities required to define project tasks and establish a workable project schedule
- Techniques for ensuring quality and controlling changes

State of problem

“I've visited dozens of commercial shops, both good and bad, and I've observed scores of data processing managers, again, both good and bad. Too often, I've watched in horror as these managers futilely struggled through nightmarish projects, squirmed under impossible deadlines, or delivered systems that outraged their users and went on to devour huge chunks of maintenance time.”

[Page-Jones, M., *Practical Project Management*, Dorset House, 1985]

Part I

Key concepts & principles

Key concepts

- **The People**
- **The Product**
- **The Process**
- **The Project**

People - Players

- **Senior managers** who define the business issues
- **Project (technical) managers** who must plan, motivate, organize, and control the practitioners who do software work.
- **Practitioners** who deliver the technical skills that are necessary to engineer a product or application.
- **Customers** who specify the requirements for the software to be engineered
- **End-users** who interact with the software once it is released for production use.

People – Team leader

- **Motivation.** The ability to encourage (by “push or pull”) technical people to produce to their best ability.
- **Organization.** The ability to mold existing processes (or invent new ones) that will enable the initial concept to be translated into a final product.
- **Ideas or innovation.** The ability to encourage people to create and feel creative even when they must work within bounds established for a particular software product or application.

People – Software Team

A project that will require n people working for k years:

1. n individuals are assigned to m different functional tasks, relatively little combined work occurs; coordination is the responsibility of a software manager who may have other projects to be concerned with
2. n individuals are assigned to m different functional tasks ($m < n$) so that informal "teams" are established; an ad hoc team leader may be appointed; coordination among teams is the responsibility of a software manager
3. n individuals are organized into t teams; each team is assigned one or more functional tasks; each team has a specific structure that is defined for all teams working on a project; coordination is controlled by both the team and a software project manager

People – Software Team

- To achieve a high-performance team:
 - Team members must have trust in one another.
 - The distribution of skills must be appropriate to the problem.
 - Mavericks may have to be excluded from the team, if team cohesiveness is to be maintained.

Key concepts - Product

- Scope is defined by answering the following questions:
 - Context.
 - How does the software to be built fit into a larger system, product, or business context and what constraints are imposed as a result of the context?
 - Information objectives.
 - What customer-visible data objects are produced as output from the software? What data objects are required for input?
 - Function and performance.
 - What function does the software perform to transform input data into output? Are any special performance characteristics to be addressed?
- Problem decomposition, is an activity that sits at the core of software requirements analysis

Key concepts - Process

- the linear sequential model
- the prototyping model
- the incremental model
- the Spiral model
- the component-based development model
- the fourth generation techniques model

Process - Framework activities

- *Customer communication*—tasks required to establish effective requirements elicitation between developer and customer.
- *Planning*—tasks required to define resources, timelines, and other project related information.
- *Risk analysis*—tasks required to assess both technical and management risks.
- *Engineering*—tasks required to build one or more representations of the application.
- *Construction and release*—tasks required to construct, test, install, and provide user support (e.g., documentation and training).
- *Customer evaluation*—tasks required to obtain customer feedback based on evaluation of the software representations created during the engineering activity and implemented during the construction activity.

Melding the Product and the Process

Common process framework activities	Customer communication				Planning				Risk analysis				Engineering			
Software engineering tasks																
Product functions																
Text input																
Editing and formatting																
Automatic copy edit																
Page layout capability																
Automatic indexing and TOC																
File management																
Document production																

Process Decomposition

- Common Process Framework (CPF)
- Process decomposition
 - *customer communication* activity (48h – small project)
 1. Develop list of clarification issues.
 2. Meet with customer to address clarification issues.
 3. Jointly develop a statement of scope.
 4. Review the statement of scope with all concerned.
 5. Modify the statement of scope as required.

Process Decomposition

- Common Process Framework (CPF)
- Process decomposition
 - *customer communication* activity
 1. Review the customer request.
 2. Plan and schedule a formal, facilitated meeting with the customer.
 3. Conduct research to specify the proposed solution and existing approaches.
 4. Prepare a “working document” and an agenda for the formal meeting.
 5. Conduct the meeting.
 6. Jointly develop mini-specs that reflect data, function, and behavioral features of the software.
 7. Review each mini-spec for correctness, consistency, and lack of ambiguity.
 8. Assemble the mini-specs into a scoping document.
 9. Review the scoping document with all concerned.
 10. Modify the scoping document as required.

Project - What can go wrong in a project?

1. Software people don't understand their customer's needs.
2. The product scope is poorly defined.
3. Changes are managed poorly.
4. The chosen technology changes.
5. Business needs change [or are ill-defined].
6. Deadlines are unrealistic.
7. Users are resistant.
8. Sponsorship is lost [or was never properly obtained].
9. The project team lacks people with appropriate skills.
10. Managers [and practitioners] avoid best practices and lessons learned.

John Reel

Key concept - Project

- Start on the right foot.

This is accomplished by working hard (very hard) to understand the problem that is to be solved and then setting realistic objects and expectations for everyone who will be involved in the project.

- Maintain momentum.

To maintain momentum, the project manager must provide incentives to keep turnover of personnel to an absolute minimum, the team should emphasize quality in every task it performs, and senior management should do everything possible to stay out of the team's way.

Key concept - Project

- Track progress.

For a software project, progress is tracked as work products; software process and project measures can be collected and used to assess progress

- Make smart decisions.

In essence, the decisions of the project manager and the software team should be to “keep it simple.”

- Conduct a postmortem analysis.

Establish a consistent mechanism for extracting lessons learned for each project. Evaluate the planned and actual schedules, collect and analyze software project metrics, get feedback from team members and customers, and record findings in written form.

W⁵HH principle

- Why is the system being developed?
- What will be done, by When?
- Who is responsible for a function?
- Where are they organizationally located?
- How will the job be done technically and managerially?
- How much of each resource is needed?

Barry Boehm

State of Art – Project definition

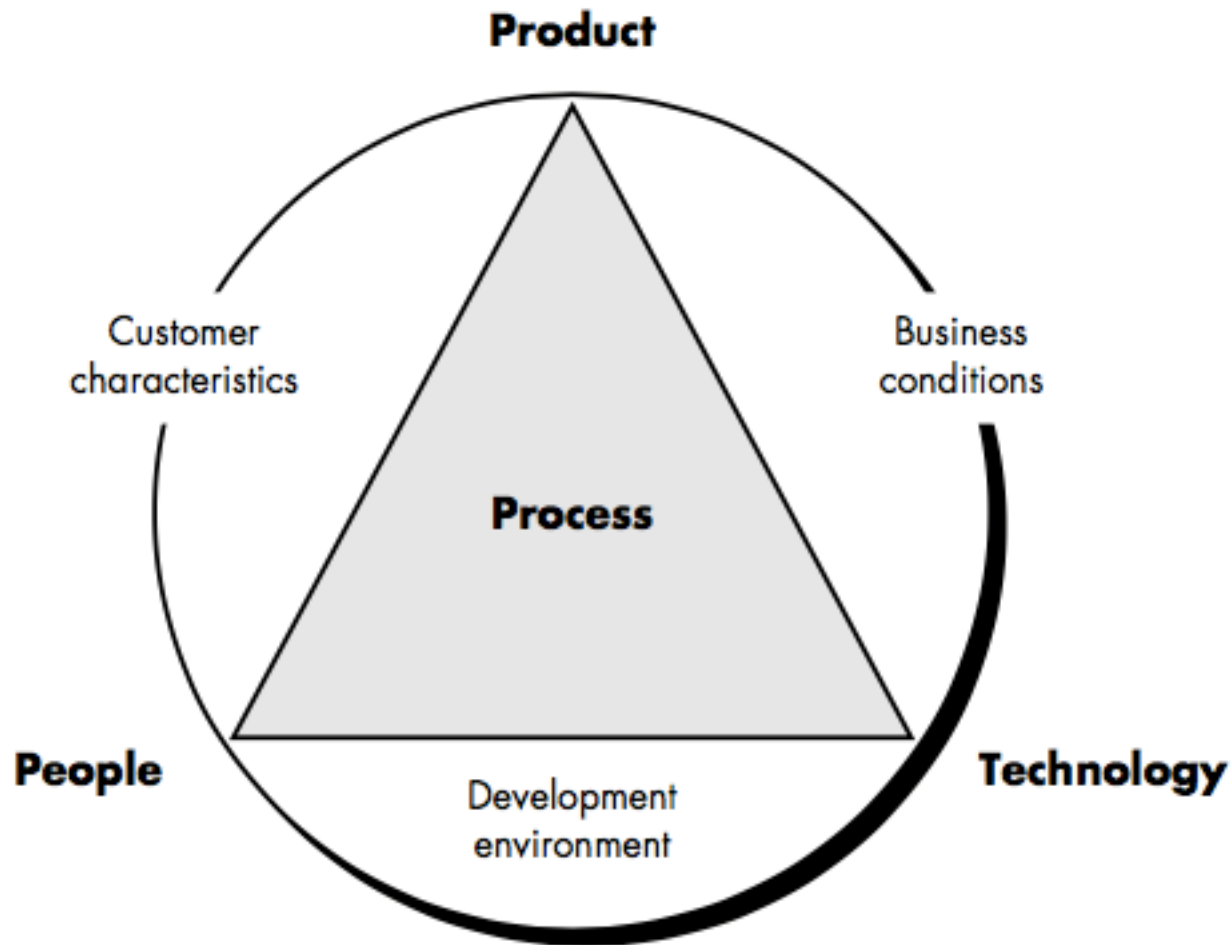
- A **project** is well-defined task, which is a collection of several operations done in order to achieve a goal (for example, software development and delivery). A Project can be characterized as:
 - Every project may has a unique and distinct goal.
 - Project is not routine activity or day-to-day operations.
 - Project comes with a start time and end time.
 - Project ends when its goal is achieved hence it is a temporary phase in the lifetime of an organization.
 - Project needs adequate resources in terms of time, manpower, finance, material and knowledge-bank.
- Software Project
 - A **Software Project** is the complete procedure of software development from requirement gathering to testing and maintenance, carried out according to the execution methodologies, in a specified period of time to achieve intended software product.



Project - Critical Practices

- **Formal risk management:** top ten risks for this project, the chance that the risk will become, the impact if it does
- **Empirical cost and schedule estimation:** the current estimated size of the application software
- **Metric-based project management:** a metrics program to give an early indication of evolving problems
- **Earned value tracking:** monthly earned value metrics?
- **Defect tracking against quality targets:** track and report the number of defects found, execution test from program inception, the number of defects currently closed / open?
- **People-aware program management:** the average staff turnover for the past three months for each of the suppliers/developers involved

Determinants for software quality & organizational effectiveness



Part II

Project Metrics and Software Measurement

Sate of problem

“When you can measure what you are speaking about and express it in numbers, you know something about it; but when you cannot measure, when you cannot express it in numbers, your knowledge is of a meager and unsatisfactory kind: it may be the beginning of knowledge, but you have scarcely, in your thoughts, advanced to the stage of a science.”

[Lord Kevin]

Goals of software measurement

- to assist in estimation, quality control, productivity assessment, and project control
- can be used by software engineers to help assess the quality of technical work products and to assist in tactical decision making as a project proceeds

Steps

- Defining a limited set of process, project, and product measures that are easy to collect (often normalized using either size-oriented or function-oriented metrics)
- The result is analyzed and compared to past averages for similar projects performed
- Trends are assessed and conclusions are generated.

Concepts

- A *measure* provides a quantitative indication of the extent, amount, dimension, capacity, or size of some attribute of a product or process
- *Measurement* is the act of determining a measure
- *Metric* is a quantitative measure of the degree to which a system, component, or process possesses a given attribute [IEEE93, *Standard Glossary of Software Engineering Terms*]
- A software *metric* relates the individual measures in some way (process, project, and product metrics)
- An *indicator* is a metric or combination of metrics that provide insight into the software process, a software project, or the product itself

Project indicators

Project indicators enable a software project manager to

- (1) assess the status of an ongoing project,
- (2) track potential risks,
- (3) uncover problem areas before they go “critical,”
- (4) adjust work flow or tasks,
- (5) evaluate the project team’s ability to control quality of software work products.

Sized-Oriented Metrics

- Size-oriented software metrics are derived by considering the *size* of the software that has been produced

Project	LOC	Effort	\$(000)	Pp. doc.	Errors	Defects	People
alpha	12,100	24	168	365	134	29	3
beta	27,200	62	440	1224	321	86	5
gamma	20,200	43	314	1050	256	64	6
•	•	•	•	•	•		
•	•	•	•	•	•		
•	•	•	•	•	•		

Sized-Oriented Metrics

- Errors per KLOC (thousand lines of code).
- Defects per KLOC.
- \$ per LOC.
- Page of documentation per KLOC.
- Efforts per person-month.
- LOC per person-month.
- \$ per page of documentation.

Function-Oriented Metrics

- Function-oriented software metrics use a measure of the functionality delivered by the application as a normalization value
- ‘functionality’ cannot be measured directly, it must be derived indirectly using other direct measures
- A measure called the ***function point*** is suggested. Function points are based on countable (direct) measures of software's information domain and assessments of software complexity

Function-Oriented Metrics

- Function points are computed by completing table following

		Weighting factor				
Measurement parameter	Count	Simple Average Complex				
Number of user inputs		×	3	4	6	=
Number of user outputs		×	4	5	7	=
Number of user inquiries		×	3	4	6	=
Number of files		×	7	10	15	=
Number of external interfaces		×	5	7	10	=
Count total						→

Function-Oriented Metrics

- Five information domain characteristics are determined and counts are provided in the appropriate table location. Information domain values are defined in the following manner:
 - Number of user inputs
 - Number of user outputs
 - Number of user inquiries
 - Number of files
 - Number of external interfaces

Function-Oriented Metrics

- **Number of user inputs.** Each user input that provides distinct application oriented data to the software is counted. Inputs should be distinguished from inquiries, which are counted separately
- **Number of user outputs.** Each user output that provides application oriented information to the user is counted. In this context output refers to reports, screens, error messages, etc. Individual data items within a report are not counted separately

Function-Oriented Metrics

- **Number of user inquiries.** An inquiry is defined as an on-line input that results in the generation of some immediate software response in the form of an on-line output. Each distinct inquiry is counted
- **Number of files.** Each logical master file (i.e., a logical grouping of data that may be one part of a large database or a separate file) is counted
- **Number of external interfaces.** All machine readable interfaces (e.g., data files on storage media) that are used to transmit information to another system are counted

Function-Oriented Metrics

- To compute function points (FP), the following relationship is used:

$$FP = \text{count total} [0.65 + 0.01 \sum (F_i)]$$

F_i ($i = 1$ to 14) are "complexity adjustment values"

Function-Oriented Metrics

- *Fi* based on responses to the following questions
 1. Does the system require reliable backup and recovery?
 2. Are data communications required?
 3. Are there distributed processing functions?
 4. Is performance critical?
 5. Will the system run in an existing, heavily utilized operational environment?
 6. Does the system require on-line data entry?

Function-Oriented Metrics

7. Does the on-line data entry require the input transaction to be built over multiple screens or operations?
8. Are the master files updated on-line?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

Each of these questions is answered using a scale that ranges from 0 (not important or applicable) to 5 (absolutely essential).

Function-Oriented Metrics

- Once function points have been calculated, they are used in a manner analogous to LOC as a way to normalize measures for software productivity, quality, and other attributes:
 - Errors per FP
 - Defects per FP
 - \$ per FP
 - Pages of documentation per FP
 - FP per person-month

Metrics for Software Quality

- The quality of a system is only as good as the requirements that describe the problem (requirement specification), the design that models the solution (design models), the code that leads to an executable program (source code), and the tests that exercise the software to uncover errors (test case), and also the project progresses

Measuring Quality Indicators

- Useful indicators:

- **Correctness.** A program must operate correctly or it provides little value to its users. Correctness is the degree to which the software performs its required function; defects per KLOC counted over a standard period of time, typically one year.
- **Maintainability.** Maintainability is the ease with which a program can be corrected if an error is encountered, adapted if its environment changes, or enhanced if the customer desires a change in requirements; a simple time-oriented metric is *mean-time-to change* (MTTC), the time it takes to analyze the change request, design an appropriate modification, implement the change, test it, and distribute the change to all users.

Measuring Quality Indicators

- Useful indicators:
 - **Integrity.** This attribute measures a system's ability to withstand attacks (both accidental and intentional) to its security. Attacks can be made on all three components of software: programs, data, and documents.
 - To measure integrity, two additional attributes must be defined: threat and security.

Measuring Quality Indicators

- Useful indicators:

- **Integrity.** The integrity of a system can then be defined as

integrity = summation [(1 – threat) (1 – security)]

where threat and security are summed over each type of attack.

***Threat** is the probability (which can be estimated or derived from empirical evidence) that an attack of a specific type will occur within a given time.*

***Security** is the probability (which can be estimated or derived from empirical evidence) that the attack of a specific type will be repelled.*

Measuring Quality Indicators

- Useful indicators:

- **Usability.** If a program is not user-friendly, it is often doomed to failure, even if the functions that it performs are valuable. Usability is an attempt to quantify user-friendliness;

Measuring Quality Indicators

- Useful indicators:

- **Usability** can be measured in terms of four characteristics:

- (1) the physical and or intellectual skill required to learn the system,

- (2) the time required to become moderately efficient in the use of the system,

- (3) the net increase in productivity (over the approach that the system replaces) measured when the system is used by someone who is moderately efficient,

- (4) a subjective assessment (sometimes obtained through a questionnaire) of users attitudes toward the system.

Defect Removal Efficiency

- A quality metric that provides benefit at both the project and process level is defect removal efficiency (DRE).
- DRE is a measure of the filtering ability of quality assurance and control activities as they are applied throughout all process framework activities.
- DRE is defined in the following manner:

$$DRE = E/(E + D)$$

where E is the number of errors found before delivery of the software to the end-user and D is the number of defects found after delivery.

The ideal value for DRE is 1

Part III

Software Project Planning

Software Project Planning

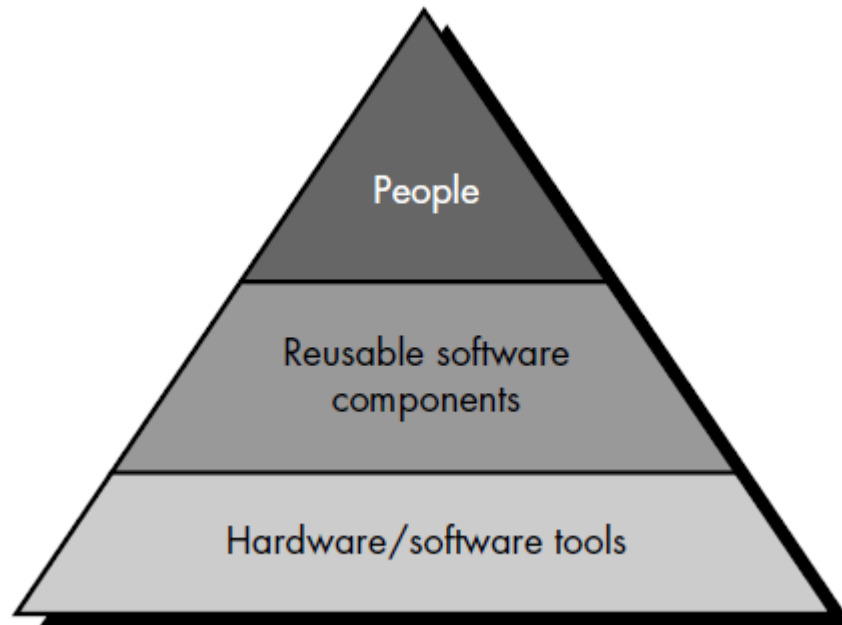
- Software project management begins with a set of activities that are collectively called *project planning*.
- Before the project can begin, the manager and the software team must estimate
 - the work to be done,
 - the resources that will be required,
 - and the time that will elapse from start to finish.
- Provide a framework that enables the manager to make reasonable estimates of resources, cost, and schedule.

Project Planning Objectives

- **Software Scope.** *Software scope* describes the data and control to be processed, function, performance, constraints, interfaces, and reliability.
 - Obtaining Information of Software Scope
 - Who is behind the request for this work?
 - Who will use the solution?
 - What will be the economic benefit of a successful solution?
 - Is there another source for the solution?, etc.
 - Feasibility
 - Once scope is understood, the software team and others must work to determine if it can be done within the dimensions just noted

Project Planning Objectives

- **Resources.** The second software planning task is estimation of the resources required to accomplish the software development effort .



Project Planning Objectives

- **Resources.** Each resource is specified with four characteristics:
 - description of the resource,
 - a statement of availability,
 - time when the resource will be required;
 - duration of time that resource will be applied

Project Planning Objectives

- **Resources.**

- **Human Resources.**

- The planner begins by evaluating scope and selecting the skills required to complete development. Both organizational position (e.g., manager, senior software engineer) and specialty (e.g., telecommunications, database, client/server) are specified.
- For relatively small projects (one person-year or less), a single individual may perform all software engineering tasks, consulting with specialists as required.
- The number of people required for a software project is determined only after an estimate of development effort (person-months) is made.

Project Planning Objectives

- **Resources.**

- **Reusable Software Resources.** Four software resource categories:

- **Off-the-shelf components.** Existing software that can be acquired from a third party or that has been developed internally for a past project
- **Full-experience components.** Existing specifications, designs, code, or test data developed for past projects that are similar to the software to be built for the current project. Members of the current software team have had full experience in the application area represented by these components.

Project Planning Objectives

- **Resources.**

- **Reusable Software Resources.** Four software resource categories (cont.):

- **Partial-experience components.** Existing specifications, designs, code, or test data developed for past projects that are related to the software to be built for the current project but will require substantial modification. Members of the current software team have only limited experience in the application area represented by these components
- **New components.** Software components that must be built by the software team specifically for the needs of the current project.

Project Planning Objectives

- **Resources.**

- **Environmental Resources.** *Software engineering environment* (SEE) incorporates hardware and software. A project planner must prescribe the time window required for hardware and software and verify that these resources will be available.

Software Project Estimation

- Software **cost** and **effort** estimation.
- Many variables - human, technical, environmental, political - can affect the ultimate cost of software and effort .
- To achieve reliable cost and effort estimates:
 1. Delay estimation until late in the project (obviously, we can achieve 100% accurate estimates after the project is complete!).
 2. Base estimates on similar projects that have already been completed.
 3. Use relatively simple decomposition techniques to generate project cost and effort estimates.
 4. Use one or more empirical models for software cost and effort estimation.

Software Project Estimation

- **Software Project Estimation Techniques**

- *Decomposition techniques* take a "divide and conquer" approach to software project estimation. By decomposing a project into major functions and related software engineering activities, cost and effort estimation can be performed in a stepwise fashion (LOC-based, FP-based Estimation).
- *Empirical estimation models* can be used to complement decomposition techniques and offer a potentially valuable estimation approach in their own right (COCOMO Model).
- *Automated estimation tools* implement one or more decomposition techniques or empirical models.

Part IV

Risk Analysis & Management

Software Risk

- Risk always involves two characteristics
 - *Uncertainty* - the risk may or may not happen; that is, there are no 100% probable risks.
 - *Loss* - if the risk becomes a reality, unwanted consequences or losses will occur.

Software Risk

- *Project risks* threaten the project plan, if project risks become real, it is likely that project schedule will slip and that costs will increase (budgetary, schedule, personnel, resource, customer and requirements problems).
- *Technical risks* threaten the quality and timeliness of the software to be produced. If a technical risk becomes a reality, implementation may become difficult or impossible (design, implementation, interface, verification, and maintenance problems).
- *Business risks* threaten the viability of the software to be built.

Software Risk

- *Risk identification* is a systematic attempt to specify threats to the project plan (estimates, schedule, resource loading, ...)
- *Risk item checklist*
 - *Product size*—risks associated with the overall size of the software to be built or modified.
 - *Business impact*—risks associated with constraints imposed by management for the marketplace
 - *Customer characteristics*—risks associated with the sophistication of the customer and the developer's ability to communicate with the customer in a timely manner.

Software Risk

- *Risk item checklist (cont.)*

- *Process definition*—risks associated with the degree to which the software process has been defined and is followed by the development organization
- *Development environment*—risks associated with the availability and quality of the tools to be used to build the product
- *Technology to be built*—risks associated with the complexity of the system to be built and the "newness" of the technology that is packaged by the system
- *Staff size and experience*—risks associated with the overall technical and project experience of the software engineers who will do the work.

Software Risk

- *Risk projection (called risk estimation)*, attempts to rate each risk in two ways
 - the likelihood or probability that the risk is real
 - the consequences of the problems associated with the risk, should it occur.

Software Risk

- *Risk projection*
 - Developing a Risk Table

Risks	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse than planned	PS	70%	2	
End-users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet expectations	TE	30%	1	
Lack of training on tools	DE	80%	3	
Staff inexperienced	ST	30%	2	
Staff turnover will be high	ST	60%	2	
•				
•				
•				

Impact values:
1—catastrophic
2—critical
3—marginal
4—negligible

Software Risk

- *Risk Refinement*

- To refine the risk into a set of more detailed risks
- Represent the risk in *condition-transition-consequence* (CTC)

“Given that <condition> then there is concern that (possibly) <consequence>.”

Then <condition> will be refined in <subcondition 1>, <subcondition 2>, <subcondition 3>,...

Software Risk

Risk information sheet			
Risk ID: P02-4-32	Date: 5/9/02	Prob: 80%	Impact: high
Description: Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.			
Refinement/context: Subcondition 1: Certain reusable components were developed by a third party with no knowledge of internal design standards. Subcondition 2: The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components. Subcondition 3: Certain reusable components have been implemented in a language that is not supported on the target environment.			
Mitigation/monitoring: 1. Contact third party to determine conformance with design standards. 2. Press for interface standards completion; consider component structure when deciding on interface protocol. 3. Check to determine number of components in subcondition 3 category; check to determine if language support can be acquired.			
Management/contingency plan/trigger: RE computed to be \$20,200. Allocate this amount within project contingency cost. Develop revised schedule assuming that 18 additional components will have to be custom built; allocate staff accordingly. Trigger: Mitigation steps unproductive as of 7/1/02			
Current status: 5/12/02: Mitigation steps initiated.			
Originator: D. Gagne		Assigned: B. Laster	

Part V

Project Scheduling and Tracking

Project Scheduling

- *Software project scheduling* is an activity that distributes estimated effort across the planned project duration by allocating the effort to specific software engineering tasks.
- *A macroscopic schedule* is refined into a *detailed schedule*.
- Effort Distribution: *40–20–40 rule*.

Project Scheduling

- ***Defining a task set***

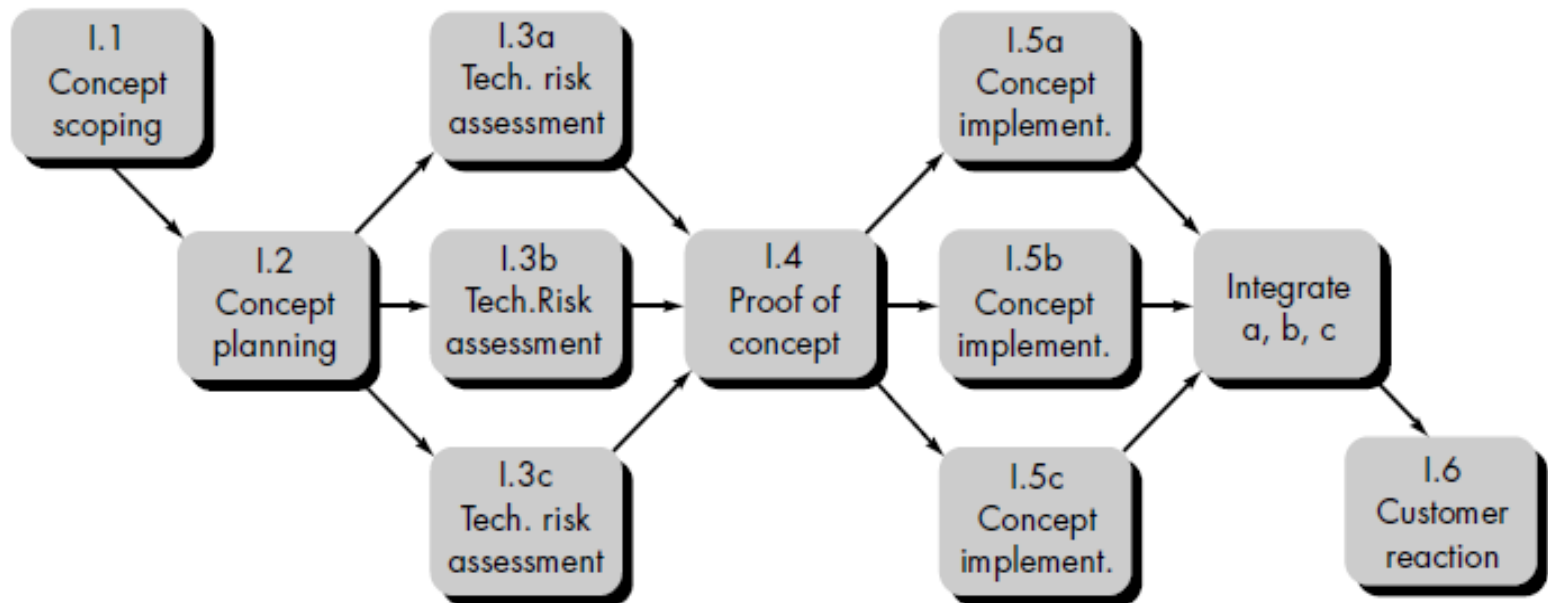
- A *task set* is a collection of software engineering work tasks, milestones, and deliverables that must be accomplished to complete a particular project.

- ***Defining a task network***

- A *task network (activity network)* is a graphic representation of the task flow for a project.

Project Scheduling

- A task network example for concept development



Three I.5 tasks are applied in parallel to 3 different concept functions

Project Scheduling

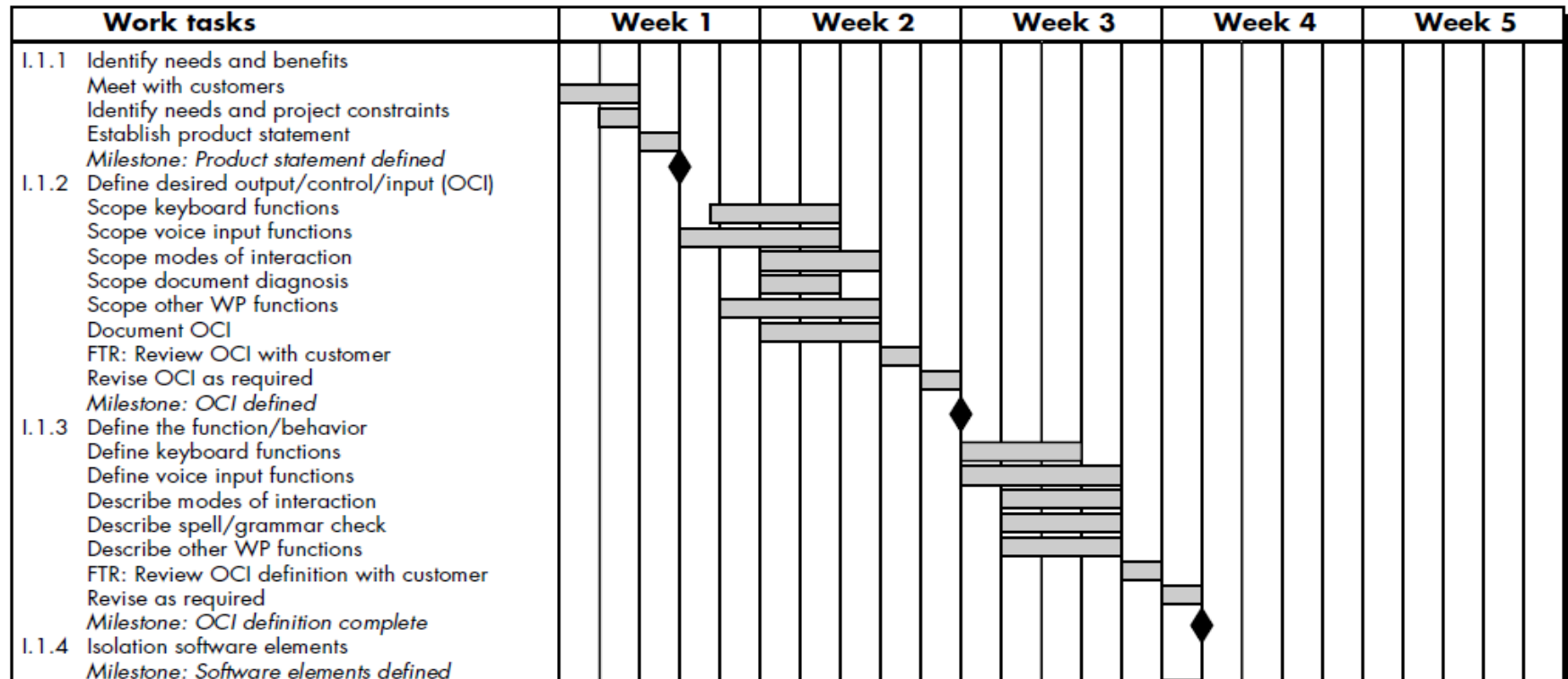
● *Scheduling*

- Two methods: *Program evaluation and review technique* (PERT) and *critical path method* (CPM).
 - Driven by: estimates of effort, a decomposition of the product function, the selection of the appropriate process model and task set, decomposition of tasks.
1. Determine the *critical path*—the chain of tasks that determines the duration of the project;
 2. Establish “most likely” time estimates for individual task by applying statistical models;
 3. Calculate “boundary times” that define a time “window” for a particular task.

Project Scheduling

● Timeline chart (*Gantt chart*)

- begins with a set of tasks (the work breakdown structure)
- effort, duration, and start date are then input for each task (may assign specific individuals).



Project Scheduling

● Project Table

- listing of all project tasks, their planned and actual start- and end-dates, and a variety of related information
- project tables enable the project manager to track progress.

Work tasks	Planned start	Actual start	Planned complete	Actual complete	Assigned person	Effort allocated	Notes
I.1.1 Identify needs and benefits							
Meet with customers	wk1, d1	wk1, d1	wk1, d2	wk1, d2	BLS	2 p-d	
Identify needs and project constraints	wk1, d2	wk1, d2	wk1, d2	wk1, d2	JPP	1 p-d	
Establish product statement	wk1, d3	wk1, d3	wk1, d3	wk1, d3	BLS/JPP	1 p-d	
Milestone: Product statement defined	wk1, d3	wk1, d3	wk1, d3	wk1, d3			Scoping will require more effort/time
I.1.2 Define desired output/control/input (OCI)							
Scope keyboard functions	wk1, d4	wk1, d4	wk2, d2		BLS	1.5 p-d	
Scope voice input functions	wk1, d3	wk1, d3	wk2, d2		JPP	2 p-d	
Scope modes of interaction	wk2, d1		wk2, d3		MLL	1 p-d	
Scope document diagnostics	wk2, d1		wk2, d2		BLS	1.5 p-d	
Scope other WP functions	wk1, d4	wk1, d4	wk2, d3		JPP	2 p-d	
Document OCI	wk2, d1		wk2, d3		MLL	3 p-d	
FTR: Review OCI with customer	wk2, d3		wk2, d3		all	3 p-d	
Revise OCI as required	wk2, d4		wk2, d4		all	3 p-d	
Milestone: OCI defined	wk2, d5		wk2, d5				

Tracking the Schedule

- Conducting periodic project status meetings in which each team member reports progress and problems.
- Evaluating the results of all reviews conducted throughout the software engineering process.
- Determining whether formal project milestones have been accomplished by the scheduled date.
- Comparing actual start-date to planned start-date for each project task listed in the project table.
- Meeting informally with practitioners to obtain their subjective assessment of progress to date and problems on the horizon.

Principles guide software project scheduling

- Compartmentalization
 - project ~ manageable activities and tasks
 - both the product and the process are decomposed
- Interdependency
 - tasks occur in sequence while others can occur in parallel
- Time allocation
 - task to be scheduled must be allocated some number of work units (e.g., person-days of effort).
 - each task must be assigned a start date and a completion date
- Effort validation
 - defined number of staff members.
 - project manager must ensure that no more than the allocated number of people have been scheduled at any given time
- Defined responsibilities.
 - Every task that is scheduled should be assigned to a specific team member.
- Defined outcomes.
 - Every task that is scheduled should have a defined out- come
- Defined milestones
 - Every task or group of tasks should be associated with a project milestone

Project Plan

- ***Software Project Plan***

- (1) communicate scope and resources to software management, technical staff, and the customer;
- (2) define risks and suggest risk aversion techniques;
- (3) define cost and schedule for management review;
- (4) provide an overall approach to software development for all people associated with the project;
- (5) outline how quality will be ensured and change will be managed.

Part VI

SOFTWARE QUALITY ASSURANCE

Quick look

● WHAT

- (1) explicitly define what is meant when you say “software quality,”
- (2) create a set of activities that will help ensure that every software engineering work product exhibits high quality,
- (3) perform quality assurance activities on every software project,
- (4) use metrics to develop strategies for improving your software process and, as a consequence, the quality of the end product.

● WHO

- Everyone involved in the software engineering process

● WHY

- You can do it right, or you can do it over again

● WORKPRODUCT

- Software Quality Assurance Plan
- formal technical review summary report

Quality concepts

- Quality
 - *American Heritage Dictionary* defines *quality* as “a characteristic or attribute of something.”
- *Quality of design*
 - the characteristics that designers specify for an item; the product is manufactured according to specifications.
- *Quality of conformance*
 - the degree to which the design specifications are followed during manufacturing.
 - User satisfaction = compliant product + good quality + delivery within budget and schedule
- DeMarco [DEM99] states: “A *product’s quality* is a function of how much it changes the world for the better.”

Quality Control

- *Quality control* involves the series of inspections, reviews, and tests used throughout the software process to ensure each work product meets the requirements placed upon it.
- Quality control includes a feedback loop to the process that created the work product. The combination of measurement and feedback allows us to tune the process when the work products created fail to meet their specifications. This approach views quality control as part of the manufacturing process.
- Quality control activities may be fully automated, entirely manual, or a combination of automated tools and human interaction.
- A key concept of quality control is that all work products have defined, measurable specifications to which we may compare the output of each process. The feedback loop is essential to minimize the defects produced.

Quality Assurance

- *Quality assurance* consists of the auditing and reporting functions of management.
- The goal of quality assurance is to provide management with the data necessary to be informed about product quality, thereby gaining insight and confidence that product quality is meeting its goals.
- Of course, if the data provided through quality assurance identify problems, it is management's responsibility to address the problems and apply the necessary resources to resolve quality issues.

SOFTWARE QUALITY ASSURANCE

- "Every program does something right, it just may not be the thing that we want it to do."
 - *Software quality* is defined: Conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software.
1. Software requirements are the foundation from which quality is measured. Lack of conformance to requirements is lack of quality.
 2. Specified standards define a set of development criteria that guide the manner in which software is engineered. If the criteria are not followed, lack of quality will almost surely result.
 3. A set of implicit requirements often goes unmentioned (e.g., the desire for ease of use and good maintainability). If software conforms to its explicit requirements but fails to meet implicit requirements, software quality is suspect.

SQA Activities

- **Prepares an SQA plan for a project.** The plan is developed during project planning and is reviewed by all interested parties. Quality assurance activities performed by the software engineering team and the SQA group are governed by the plan. The plan identifies
 - evaluations to be performed
 - audits and reviews to be performed
 - standards that are applicable to the project
 - procedures for error reporting and tracking
 - documents to be produced by the SQA group
 - amount of feedback provided to the software project team
- **Participates in the development of the project's software process description.** The software team selects a process for the work to be performed. The SQA group reviews the process description for compliance with organizational policy, internal software standards, externally imposed standards (e.g., ISO-9001), and other parts of the software project plan.
- **Reviews software engineering activities to verify compliance with the defined software process.** The SQA group identifies, documents, and tracks deviations from the process and verifies that corrections have been made.
- **Audits designated software work products to verify compliance with those defined as part of the software process.** The SQA group reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made; and periodically reports the results of its work to the project manager.
- **Ensures that deviations in software work and work products are documented and handled according to a documented procedure.** Deviations may be encountered in the project plan, process description, applicable standards, or technical work products.
- **Records any noncompliance and reports to senior management.** Noncompliance items are tracked until they are resolved.